

```
# =====
# EXPERIMENT 3 - LOGISTIC REGRESSION FROM SCRATCH
# Part 1: Baseline Logistic Regression
# =====

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
# =====
# 1. LOAD DATASET
# =====

df = pd.read_csv("emails.csv")

print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

print("\nClass distribution:")
print(df["spam"].value_counts())
```

Dataset shape: (5728, 2)

First 5 rows:

|   | text                                              | spam |
|---|---------------------------------------------------|------|
| 0 | Subject: naturally irresistible your corporate... | 1    |
| 1 | Subject: the stock trading gunslinger fanny i...  | 1    |
| 2 | Subject: unbelievable new homes made easy im ...  | 1    |
| 3 | Subject: 4 color printing special request add...  | 1    |
| 4 | Subject: do not have money , get software cds ... | 1    |

Class distribution:

```
spam
0    4360
1    1368
```

Name: count, dtype: int64

```
# =====
# 2. SEPARATE INPUT (X) AND TARGET (y)
# =====

X_text = df["text"]
y = df["spam"].values
```

```
# =====
# 3. TRAIN-TEST SPLIT
# =====

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train_text))
print("Testing samples:", len(X_test_text))
```

Training samples: 4582  
Testing samples: 1146

```
# =====
# 4. TEXT → NUMERICAL FEATURES
# =====
```

```

vectorizer = CountVectorizer()

X_train = vectorizer.fit_transform(X_train_text).toarray()
X_test = vectorizer.transform(X_test_text).toarray()

print("\nNumber of features:", X_train.shape[1])
print("Training feature matrix:", X_train.shape)
print("Testing feature matrix:", X_test.shape)

```

```

Number of features: 33772
Training feature matrix: (4582, 33772)
Testing feature matrix: (1146, 33772)

```

```

# =====
# 5. LOGISTIC REGRESSION FROM SCRATCH
# =====

class LogisticRegressionScratch:

    def __init__(self, learning_rate=0.01, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.loss_history = []

    # -----
    # Sigmoid function
    # -----

    def sigmoid(self, z):
        # Clip values to avoid numerical overflow
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    # -----
    # Binary Cross-Entropy Loss
    # -----

    def compute_loss(self, y, y_pred):

        epsilon = 1e-15

        y_pred = np.clip(
            y_pred,
            epsilon,
            1 - epsilon
        )

        loss = -np.mean(
            y * np.log(y_pred)
            + (1 - y) * np.log(1 - y_pred)
        )

        return loss

    # -----
    # Training
    # -----

    def fit(self, X, y):

        n_samples, n_features = X.shape

        # Initialize weights and bias
        self.weights = np.zeros(n_features)
        self.bias = 0

        for epoch in range(self.epochs):

            # -----
            # Forward propagation
            # -----

            linear_output = np.dot(X, self.weights) + self.bias

            y_pred = self.sigmoid(linear_output)

```

```

# -----
# Calculate loss
# -----

loss = self.compute_loss(y, y_pred)

self.loss_history.append(loss)

# -----
# Calculate gradients
# -----

dw = (1 / n_samples) * np.dot(
    X.T,
    (y_pred - y)
)

db = (1 / n_samples) * np.sum(
    y_pred - y
)

# -----
# Update parameters
# -----

self.weights -= self.learning_rate * dw
self.bias -= self.learning_rate * db

# Print progress
if (epoch + 1) % 100 == 0:
    print(
        f"Epoch {epoch + 1}/{self.epochs}, "
        f"Loss: {loss:.4f}"
    )

# -----
# Predict probabilities
# -----

def predict_proba(self, X):

    linear_output = np.dot(
        X,
        self.weights
    ) + self.bias

    return self.sigmoid(linear_output)

# -----
# Predict class
# -----

def predict(self, X):

    probabilities = self.predict_proba(X)

    return (probabilities >= 0.5).astype(int)

```

```

# =====
# 6. CREATE AND TRAIN MODEL
# =====

model = LogisticRegressionScratch(
    learning_rate=0.01,
    epochs=100
)

model.fit(X_train, y_train)

```

Epoch 100/100, Loss: 0.2775

```

# =====
# 7. MAKE PREDICTIONS
# =====

```

```
y_pred = model.predict(X_test)
```

```
# =====
# 8. EVALUATION
# =====

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

cm = confusion_matrix(
    y_test,
    y_pred
)
```

```
# =====
# 9. PRINT RESULTS
# =====

print("\n" + "=" * 50)
print("BASELINE LOGISTIC REGRESSION RESULTS")
print("=" * 50)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Not Spam", "Spam"],
        zero_division=0
    )
)
```

```
=====
BASELINE LOGISTIC REGRESSION RESULTS
=====
Accuracy : 0.9180
Precision : 0.9688
Recall : 0.6788
F1-Score : 0.7983

Confusion Matrix:
[[866   6]
 [ 88 186]]

Classification Report:

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Not Spam     | 0.91      | 0.99   | 0.95     | 872     |
| Spam         | 0.97      | 0.68   | 0.80     | 274     |
| accuracy     |           |        | 0.92     | 1146    |
| macro avg    | 0.94      | 0.84   | 0.87     | 1146    |
| weighted avg | 0.92      | 0.92   | 0.91     | 1146    |

